

LAPORAN PRAKTIKUM PEKAN 2
PEMOGRAMAN BERORIENTASI OBJEK

“Relasi Antar Objek (Class Diagram & Asosiasi)”

disusun Oleh:

Nabil Fikri (2511533011)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M. Dawi Syauqi



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

ISI PRAKTIKUM

A. Langkah Kerja Praktikum

Pada pekan 2 ini, mahasiswa melanjutkan class pada pekan 1 kemaren, memperbaiki method **setoTunai** dan **TarikTunai** method dan menambahkan satu class baru yaitu class **Transaksi**.

1. Membuat Class Transaksi

Dibuat file baru bernama Transaksi. Class ini memiliki tiga atribut yaitu idTransaksi (String), jenis (String), dan nominal (double). Jenis transaksi terdiri dari dua nilai: "Kredit" untuk penyetoran dan "Debit" untuk penarikan

```
1. package Pekan2;
2.
3. public class Transaksi {
4.
5.     String idTransaksi;
6.     String jenis;
7.     double nominal;
8.
9.     public Transaksi(String id, String Jenis, double nominal) {
10.         this.idTransaksi = id;
11.         this.jenis = Jenis;
12.         this.nominal = nominal;
13.     }
14.
15.     public void cetakDetail() {
16.         System.out.println("ID " + idTransaksi + " | Jenis : " + jenis +
17.         " | Nominal : Rp." + nominal);
18.     }
19. }
```

Class Transaksi merepresentasikan satu data transaksi yang terjadi pada sebuah rekening. Atribut yang dimiliki ada tiga: idTransaksi untuk menyimpan kode unik transaksi, jenis untuk membedakan apakah transaksi berupa "Kredit" (setor) atau "Debit" (tarik), dan nominal untuk menyimpan jumlah uang yang ditransaksikan.

Constructor Transaksi() menerima ketiga nilai tersebut sebagai parameter dan mengisinya ke masing-masing atribut. Objek ini dibuat secara otomatis oleh class Rekening setiap kali transaksi berhasil dilakukan.

Method yang tersedia hanya satu: cetakDetail() yang mencetak satu baris informasi transaksi ke layar dalam format ID, jenis, dan nominal — digunakan oleh cetakMutasi() di class Rekening saat menampilkan riwayat.

2. Menambahkan Relasi pada class Rekening

Dibuka kembali class Rekening dari Praktikum 1. Ditambahkan atribut baru bertipe ArrayList<Transaksi> bernama riwayatTransaksi untuk merepresentasikan relasi **has-a** (satu rekening memiliki banyak transaksi). ArrayList ini wajib diinisialisasi di dalam constructor agar tidak terjadi NullPointerException.

```
1. package Pekan2;
2. import java.util.ArrayList;
3. import java.util.*;
4.
5. public class Rekening {
6.
7.     String nomorRekening;
8.     String namaPemilik;
9.     double saldo;
10.
11.     ArrayList<Transaksi> riwayatTransaksi;
12.
13.     public Rekening(String nomor, String nama, double saldoAwal) {
14.         this.nomorRekening = nomor;
15.         this.namaPemilik = nama;
16.         this.saldo = saldoAwal;
17.
18.         this.riwayatTransaksi = new ArrayList<>();
19.
20.         System.out.println("Rekening atas nama " + namaPemilik + "
    berhasil dibuat dengan saldo Rp" + saldo);
21.     }
22.
23.     public void setorTunai(double nominal) {
24.         if (nominal > 0) {
```

```

25.         saldo += nominal;
26.
27.         String idTrx = "TRX-S-" + System.currentTimeMillis();
28.         Transaksi trxBaru = new Transaksi(idTrx, "Kredit",
nominal);
29.         riwayatTransaksi.add(trxBaru);
30.
31.         System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini : Rp" + saldo);
32.     } else {
33.         System.out.println("Gagal : Nominal setor harus lebih dari
0!");
34.     }
35. }
36. public void tarikTunai(double nominal) {
37.     if (nominal >= 10000) {
38.         if (nominal > saldo) {
39.
40.             System.out.println("Transaksi Gagal: Saldo tidak
mencukupi. Saldo saat ini : Rp" + saldo );
41.         } else {
42.             saldo -= nominal;
43.
44.             String idTrx = "TRX-T-" + System.currentTimeMillis();
45.             Transaksi trxBaru = new Transaksi(idTrx, "Debit",
nominal);
46.             riwayatTransaksi.add(trxBaru);
47.
48.             System.out.println("Tarik tunai Rp" + nominal + " berhasil.
Saldo saat ini : Rp" + saldo);
49.         }
50.     } else {
51.         System.out.println("Transaksi Gagal : Minimal nominal penarikan
Rp10.000");
52.     }
53. }
54.
55. public void cetakMutasi() {
56.     if (riwayatTransaksi.isEmpty()) {
57.         System.out.println("Belum ada transaksi pada rekening
ini");
58.     } else {
59.         System.out.println("--- MUTASI REKENING ---");
60.         int totalTrx = riwayatTransaksi.size();
61.
62.         if (totalTrx > 3) {
63.             System.out.println("(Menampilkan 3 transaksi
terbaru dari " + totalTrx + " total transaksi)");
64.             for (int total = totalTrx - 3; total < totalTrx;
total++) {
65.                 riwayatTransaksi.get(total).cetakDetail();
66.             }
67.         } else {
68.             for (Transaksi trx : riwayatTransaksi) {
69.                 trx.cetakDetail();

```

```

70.         }
71.     }
72. }
73. }
74.
75. public void cekInformasi() {
76.     System.out.println("--- INFO REKENING ---");
77.     System.out.println("No. Rekening : " + nomorRekening);
78.     System.out.println("Nama Pemilik : " + namaPemilik);
79.     System.out.println("Saldo Akhir : Rp" + saldo);
80.     System.out.println("-----");
81. }
82. }

```

Method `setorTunai` diperbarui agar setiap kali penyetoran berhasil, secara otomatis dibuat objek Transaksi baru dengan jenis "Kredit" dan ID bertanda prefiks "TRX-S-", lalu disimpan ke dalam `riwayatTransaksi`.

Class `Rekening` merepresentasikan sebuah rekening bank sederhana dengan kemampuan mencatat riwayat transaksi. Atribut yang dimiliki ada empat: `nomorRekening` dan `namaPemilik` sebagai identitas rekening, `saldo` untuk menyimpan saldo terkini, serta `riwayatTransaksi` bertipe `ArrayList<Transaksi>` untuk menampung seluruh transaksi yang pernah terjadi.

Constructor `Rekening()` menginisialisasi keempat atribut tersebut sekaligus membuat `ArrayList` kosong yang siap menampung transaksi.

Method-method yang tersedia:

- `setorTunai()` — menambah saldo jika nominal lebih dari 0, lalu otomatis membuat dan menyimpan objek Transaksi berjenis "Kredit" ke dalam `riwayatTransaksi`.
- `tarikTunai()` — mengurangi saldo dengan dua lapis validasi: nominal minimal Rp10.000 dan saldo harus mencukupi. Jika berhasil, mencatat objek Transaksi berjenis "Debit".
- `cetakMutasi()` — menampilkan riwayat transaksi. Jika kosong, mencetak pesan tidak ada transaksi. Jika lebih dari 3 transaksi, hanya menampilkan 3 terbaru; jika 3 atau kurang, menampilkan semua.

- cekInformasi() — menampilkan ringkasan data rekening berupa nomor, nama pemilik, dan saldo akhir.

3. Class Main

```

1 package Pekan2;
2 import java.util.ArrayList;
3
4
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         ArrayList<Rekening> daftarRekening = new ArrayList<>();
10        Rekening akunAktif = null;
11        boolean isRunning = true;
12
13        System.out.println("=== SISTEM PERBANKAN MINI ===");
14
15        while (isRunning) {
16            System.out.println("\nMenu Utama : ");
17            System.out.println("1. Buka Rekening Baru");
18            System.out.println("2. Setor Tunai");
19            System.out.println("3. Tarik Tunai");
20            System.out.println("4. Cek Informasi Rekening");
21            System.out.println("5. Ganti Akun");
22            System.out.println("6. Cetak Mutasi (Riwayat)");
23            System.out.println("0. Keluar");
24            System.out.print("Pilih Menu : ");
25
26            int pilihan = input.nextInt();
27            input.nextLine();
28
29            switch (pilihan) {
30                case 1 :
31                    System.out.print("Masukkan No Rekening : ");
32                    String no = input.nextLine();
33                    System.out.print("Masukkan Nama Pemilik : ");
34                    String nama = input.nextLine();
35                    System.out.print("Masukkan Saldo Awal : ");
36                    double saldo = input.nextDouble();
37                    input.nextLine();
38
39                    Rekening rekeningBaru = new Rekening(no, nama, saldo);
40                    daftarRekening.add(rekeningBaru);
41                    akunAktif = rekeningBaru;
42                    break;
43
44                case 2 :
45                    if (akunAktif == null) {
46                        System.out.println("Error: Mohon maaf, anda belum memiliki nomor rekening!");
47                    } else {
48                        System.out.print("Masukkan nominal setor: ");
49                        double setor = input.nextDouble();
50                        akunAktif.setorTunai(setor);
51                    }
52                    break;
53
54                case 3 :
55                    if (akunAktif == null) {
56                        System.out.println("Error: Mohon maaf, anda belum memiliki nomor rekening!");
57                    } else {
58                        System.out.print("Masukkan nominal tarik: ");
59                        double tarik = input.nextDouble();
60                        akunAktif.tarikTunai(tarik);
61                    }
62                    break;
63
64                case 4 :
65                    if (akunAktif == null) {
66                        System.out.println("Error: Anda belum membuka rekening!");
67                    } else {
68                        akunAktif.cekInformasi();
69                    }
70                    break;
71
72                case 5 :
73                    if (daftarRekening.isEmpty()) {
74                        System.out.println("Error: Belum ada rekening yang terdaftar!");
75                    }
76                    break;
77            }
78        }
79    }
80 }

```

```

78     }
79     System.out.print("Masukkan nomor rekening yang ingin diaktifkan: ");
80     String noDicari = input.nextLine();
81
82     Rekening hasilCari = null;
83     for (Rekening r : daftarRekening) {
84         if (r.nomorRekening.equals(noDicari)) {
85             hasilCari = r;
86             break;
87         }
88     }
89
90     if (hasilCari != null) {
91         akunAktif = hasilCari;
92         System.out.println("Berhasil! Akun aktif sekarang: " + akunAktif.nomorRekening + " (" + akunAktif.namaPemilik + ")");
93     } else {
94         System.out.println("Error: Nomor rekening tidak ditemukan!");
95     }
96     break;
97
98     case 6 :
99         if (akunAktif == null) {
100             System.out.println("Error: Anda belum membuka rekening!");
101         } else {
102             akunAktif.cetakMutasi();
103         }
104         break;
105
106     case 0 :
107         isRunning = false;
108         System.out.println("Sistem ditutup. Terima Kasih!");
109         break;
110
111     default:
112         System.out.println("Pilihan tidak valid!");
113     }
114 }
115 input.close();
116 }

```

Class Main adalah titik masuk program yang mengelola interaksi pengguna melalui menu berbasis teks dalam sebuah loop while.

Sebelum loop dimulai, disiapkan tiga variabel utama: `daftarRekening` untuk menampung semua objek `Rekening` yang dibuat, `akunAktif` untuk melacak rekening yang sedang digunakan, dan `isRunning` sebagai pengendali loop.

Output :

```

=== SISTEM PERBANKAN MINI ===

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar

```

Terdapat 7 case pada class ini :

Case 1 : Buka Rekening Baru

Pengguna diminta memasukkan nomor rekening, nama pemilik, dan saldo awal. Dari data tersebut dibuat objek Rekening baru yang langsung ditambahkan ke daftarRekening dan dijadikan akunAktif.

Output :

```
Pilih Menu : 1
Masukkan No Rekening : 123456789
Masukkan Nama Pemilik : Nabil
Masukkan Saldo Awal : 100000
Rekening atas nama Nabil berhasil dibuat dengan saldo Rp100000.0
```

Case 2 : Setor Tunai

Dicek terlebih dahulu apakah akunAktif sudah ada. Jika belum, ditampilkan pesan error. Jika sudah, pengguna diminta memasukkan nominal lalu dipanggil setorTunai() pada rekening aktif.

Output :

a. Berhasil

```
Pilih Menu : 2
Masukkan nominal setor: 10000
Setor tunai Rp10000.0 berhasil. Saldo saat ini : Rp20000.0
```

b. Gagal

```
Pilih Menu : 2
Masukkan nominal setor: 0
Gagal : Nominal setor harus lebih dari 0!
```

Nilai yang diinput harus lebih besar dari 0.

c. Error

```
Menu Utama :  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
0. Keluar  
Pilih Menu : 2  
Error: Mohon maaf, anda belum memiliki nomor rekening!
```

Kasus ini terjadi jika pengguna belum membuka rekening dan langsung memilih opsi setor tunai, sistem otomatis error karena tidak memiliki nomor rekening yang dituju.

Case 3 : Tarik Tunai

Sama seperti case 2, dicek dulu apakah akunAktif tersedia. Jika ada, pengguna memasukkan nominal penarikan lalu dipanggil tarikTunai() pada rekening aktif.

Output :

a. Berhasil

```
Pilih Menu : 3  
Masukkan nominal tarik: 10000  
Tarik tunai Rp10000.0 berhasil. Saldo saat ini : Rp0.0
```

Nilai minimal dari penarikan saldo harus $\geq$ Rp.10.000

b. Gagal

```
Pilih Menu : 3  
Masukkan nominal tarik: 9999  
Transaksi Gagal : Minimal nominal penarikan Rp10.000
```

Terlihat bahwa nilai tersebut < 10.000 , akibatnya transaksi gagal.

c. Saldo tidak mencukupi

```
Pilih Menu : 3  
Masukkan nominal tarik: 120000  
Transaksi Gagal: Saldo tidak mencukupi. Saldo saat ini : Rp100000.0
```

Kasus ini terjadi jika nominal Tarik tunai lebih besar dari saldo rekening.

Case 4 : Cek Informasi Rekening

Menampilkan ringkasan data rekening aktif berupa nomor rekening, nama pemilik, dan saldo akhir dengan memanggil cekInformasi().

Output :

a. Berhasil

```
Pilih Menu : 4
--- INFO REKENING ---
No. Rekening : 123
Nama Pemilik : Nabil
Saldo Akhir : Rp100000.0
-----
```

berikut informasi dari rekening sebelumnya yang dilakukan pada case Tarik tunai.

b. Gagal

```
=== SISTEM PERBANKAN MINI ===

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu : 4
Error: Anda belum membuka rekening!
```

Case ini terjadi jika pengguna tidak terlebih dahulu membuka rekeningnya.

Case 5 : Ganti Akun

Pengguna memasukkan nomor rekening yang ingin diaktifkan. Program menelusuri daftarRekening menggunakan for-each untuk mencari kecocokan. Jika ditemukan, rekening tersebut dijadikan akunAktif baru; jika tidak, ditampilkan pesan error.

Output :

```
Pilih Menu : 5
Masukkan nomor rekening yang ingin diaktifkan: 123
Berhasil! Akun aktif sekarang: 123 (Nabil)
```

Case ini terjadi jika pengguna telah mendaftarkan beberapa rekening. Jika nomor rekening salah atau belum mendaftarkan rekening, maka output yang keluar “Error”.

Case 6 : Cetak Mutasi (Riwayat)

Memanggil cetakMutasi() pada rekening aktif untuk menampilkan riwayat transaksi yang tersimpan di riwayatTransaksi.

Output :

```
--- MUTASI REKENING ---
(Menampilkan 3 transaksi terbaru dari 5 total transaksi)
ID TRX-S-1790056793775 | Jenis : Kredit | Nominal : Rp.200000.0
ID TRX-T-1790056799685 | Jenis : Debit | Nominal : Rp.150000.0
ID TRX-S-1790056807952 | Jenis : Kredit | Nominal : Rp.90000.0
```

Jika dalam satu waktu pengguna melakukan > 3 transaksi, maka Riwayat yang akan ditampilkan 3 transaksi terakhir yang dilakukan.

Case 0 : Keluar

Mengubah nilai isRunning menjadi false sehingga loop while berhenti dan program berakhir. input.close() dipanggil setelah loop untuk menutup Scanner dan mencegah kebocoran resource.

Default : Pilihan Tidak Valid

Menangani input selain 0–6. Program mencetak pesan error dan kembali menampilkan menu tanpa menghentikan loop.

B. Tugas Praktikum

Tugas 1 : Integrasi Rekam Jejak Penarikan (30%)

- Buka kembali method tarikTunai(double nominal) di dalam file Rekening.java.
- Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai “Debit” dan kodenya menggunakan prefiks “TRX-T-”. Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

```
1. public void tarikTunai(double nominal) {
2.     if (nominal >= 10000) {
3.         if (nominal > saldo) {
4.
5.             System.out.println("Transaksi Gagal: Saldo tidak
   mencukupi. Saldo saat ini : Rp" + saldo );
6.         } else {
7.             saldo -= nominal;
8.
9.             String idTrx = "TRX-T-" + System.currentTimeMillis();
10.            Transaksi trxBaru = new Transaksi(idTrx, "Debit",
   nominal);
11.            riwayatTransaksi.add(trxBaru);
12.
13.            System.out.println("Tarik tunai Rp" + nominal + " berhasil.
   Saldo saat ini : Rp" + saldo);
14.        }
15.    } else {
16.        System.out.println("Transaksi Gagal : Minimal nominal penarikan
   Rp10.000");
17.    }
18. }
```

Method tarikTunai diperbarui dengan menambahkan validasi minimal nominal sebesar Rp10.000 dan pengecekan kecukupan saldo. Jika penarikan berhasil, dibuat objek Transaksi baru dengan jenis “**Debit**” dan ID menggunakan prefiks “TRX-T-“. Pencatatan ke riwayatTransaksi hanya dilakukan apabila saldo mencukupi.

Output :

```
Pilih Menu : 3
Masukkan nominal tarik: 9999
Transaksi Gagal : Minimal nominal penarikan Rp10.000
```

Terlihat bahwa nilai tersebut < 10.000, akibatnya transaksi gaga

Tugas 2 : Fitur Cetak Mutasi Rekening (70%)

- Buat sebuah method baru di dalam class Rekening.java dengan nama cetakMutasi().
- Di dalam method tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil method cetakDetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: "Belum ada transaksi pada rekening ini".

```
1. public void cetakMutasi() {
2.     if (riwayatTransaksi.isEmpty()) {
3.         System.out.println("Belum ada transaksi pada rekening
4.         ini");
5.     } else {
6.         System.out.println("--- MUTASI REKENING ---");
7.         int totalTrx = riwayatTransaksi.size();
8.         if (totalTrx > 3) {
9.             System.out.println("(Menampilkan 3 transaksi
10.            terbaru dari " + totalTrx + " total transaksi)");
11.             for (int total = totalTrx - 3; total < totalTrx;
12.             total++) {
13.                 riwayatTransaksi.get(total).cetakDetail();
14.             }
15.         } else {
16.             for (Transaksi trx : riwayatTransaksi) {
17.                 trx.cetakDetail();
18.             }
19.         }
20.     }
21. }
```

Dibuat method baru cetakMutasi() yang menampilkan riwayat transaksi pada rekening aktif. Jika riwayatTransaksi kosong, ditampilkan pesan bahwa belum ada transaksi. Jika jumlah transaksi melebihi 3, program hanya menampilkan 3 transaksi terbaru menggunakan perulangan for berbasis indeks. Jika 3 atau kurang, seluruh transaksi ditampilkan menggunakan perulangan **for-each** dengan memanggil cetakDetail() pada setiap objek Transaksi.

Output : (telah ditampilkan pada case diatas)

```
--- MUTASI REKENING ---
(Menampilkan 3 transaksi terbaru dari 5 total transaksi)
ID TRX-S-1790056793775 | Jenis : Kredit | Nominal : Rp.200000.0
ID TRX-T-1790056799685 | Jenis : Debit | Nominal : Rp.150000.0
ID TRX-S-1790056807952 | Jenis : Kredit | Nominal : Rp.90000.0
```

Jika dalam satu waktu pengguna melakukan > 3 transaksi, maka Riwayat yang akan ditampilkan 3 transaksi terakhir yang dilakukan.

- Buka file Main.java, tambahkan menu baru: "6. Cetak Mutasi (Riwayat)". Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

```
1. case 6 :
2.         if (akunAktif == null) {
3.             System.out.println("Error: Anda belum membuka
   rekening!");
4.         } else {
5.             akunAktif.cetakMutasi();
6.         }
7.         break;
```

Pada class Main, ditambahkan pilihan menu baru “**6. Cetak Mutasi (Riwayat)**” di dalam loop menu utama. Saat pengguna memilih opsi 6, program memeriksa apakah akunAktif sudah ada, lalu memanggil method cetakMutasi().